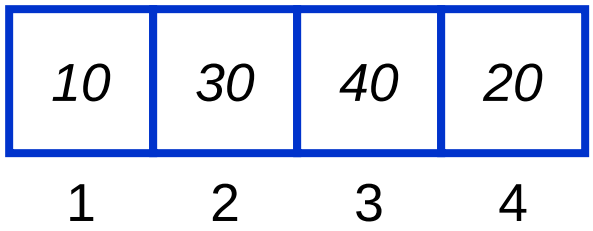
Insertion Sort

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i];
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

Illustration of Insertion Sort



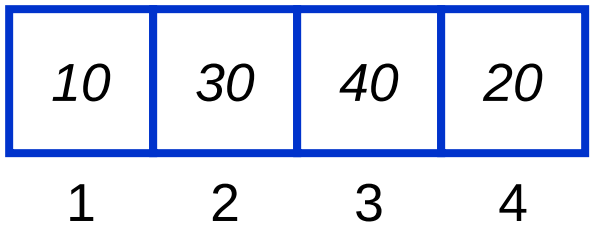
$i = \varnothing \quad j = \varnothing \quad key = \varnothing$

$A[j] = \varnothing \qquad A[j+1] = \varnothing$



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

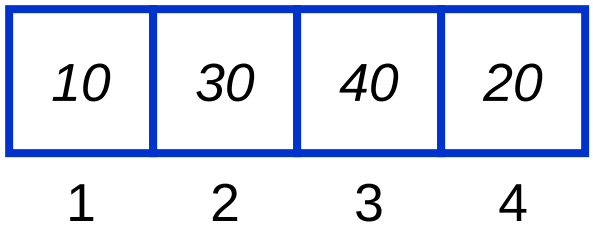


i = 2 j = ∅ key = ∅
A[j] = ∅ A[j+1] = ∅



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

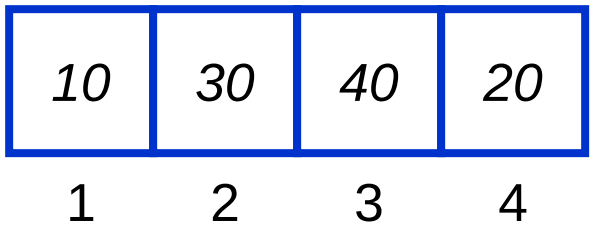


i = 2 j = ∅ key = 30
A[j] = ∅ A[j+1] = ∅



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

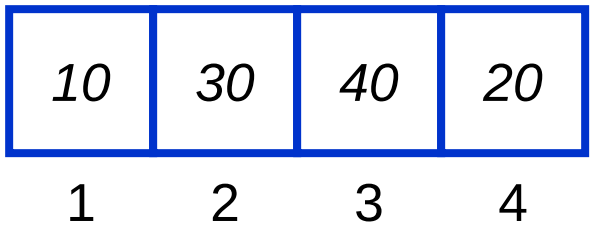


i = 2 j = ∅ key = 30
A[j] = ∅ A[j+1] = ∅



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

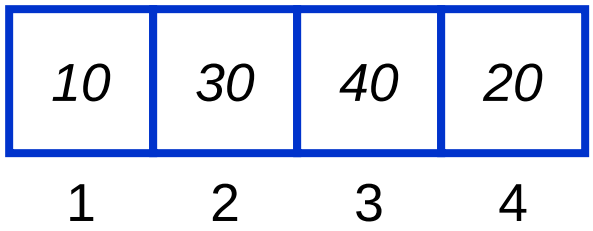


i = 2 j = 1 key = 30
A[j] = 10 A[j+1] = 30



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

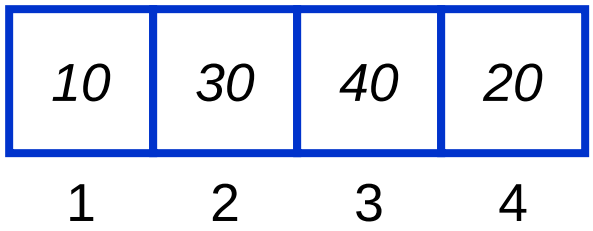
Illustration of Insertion Sort



i = 2 j = 1 key = 30
A[j] = 10 A[j+1] = 30

```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

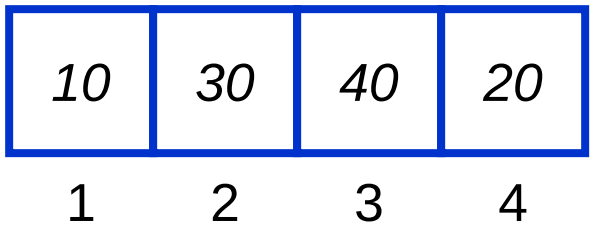
Illustration of Insertion Sort



i = 2	j = 1	key = 30
A[j] = 10		A[j+1] = 30

```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

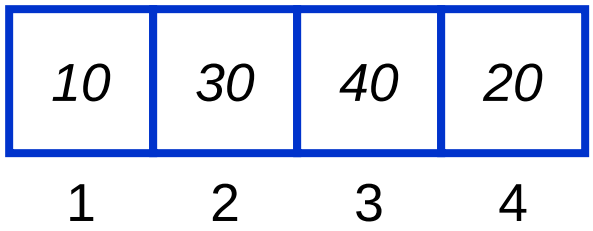


i = 3 j = 1 key = 30
A[j] = 10 A[j+1] = 30



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

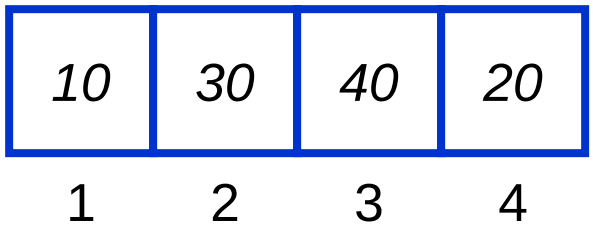


i = 3 j = 1 key = 40
A[j] = 10 A[j+1] = 30



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

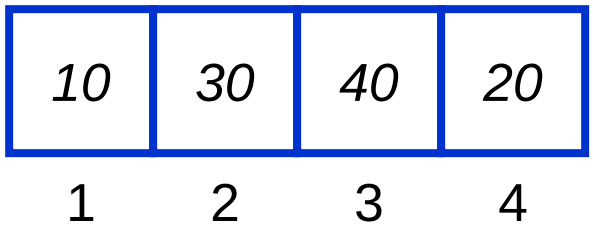


i = 3 j = 1 key = 40
A[j] = 10 A[j+1] = 30



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

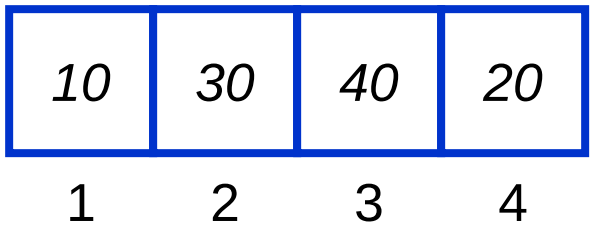


i = 3 j = 2 key = 40
A[j] = 30 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

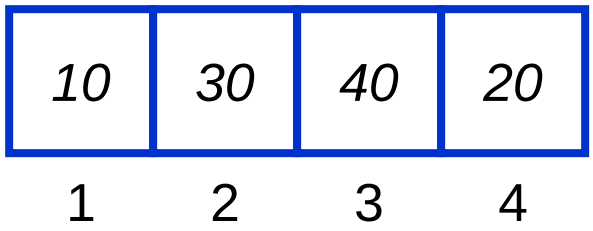
Illustration of Insertion Sort



i = 3 j = 2 key = 40
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

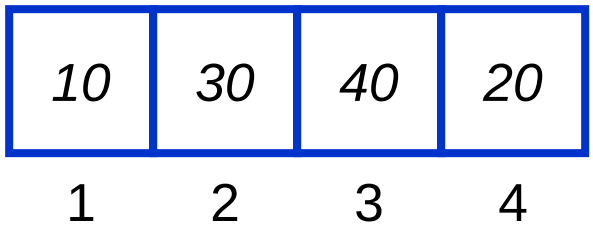
Illustration of Insertion Sort



i = 3 j = 2 key = 40
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

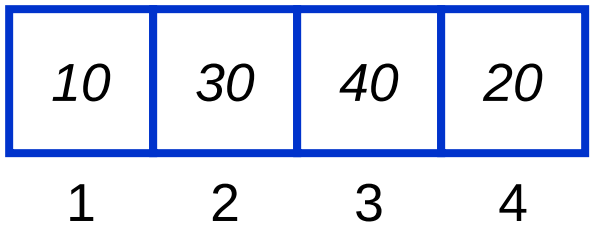


i = 4 j = 2 key = 40
A[j] = 30 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

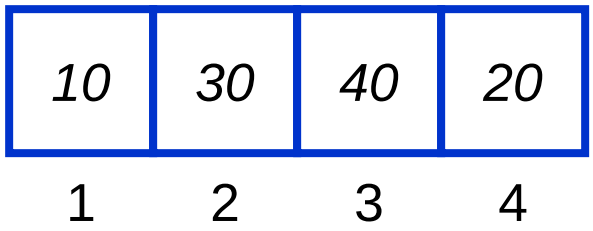


i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

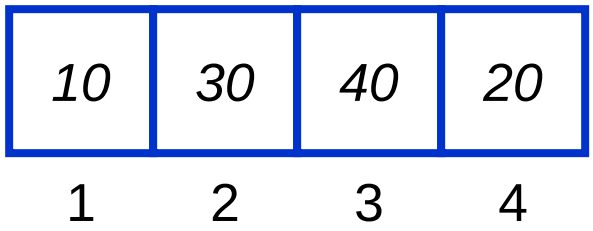


i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

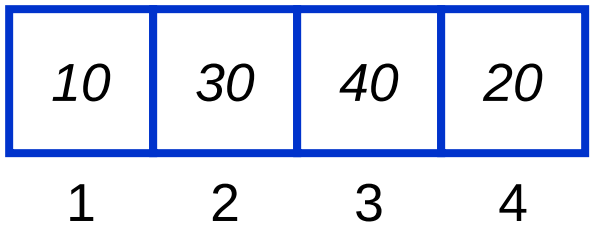


i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 20



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

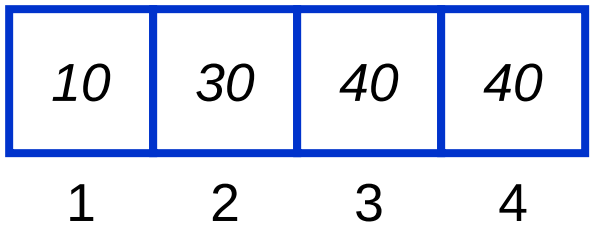


i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 20



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

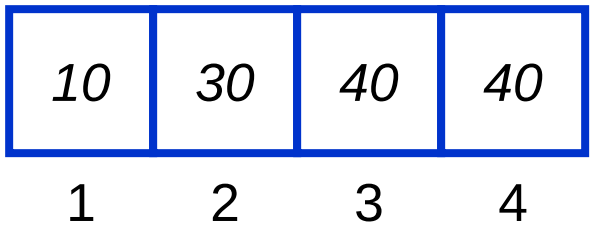


i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

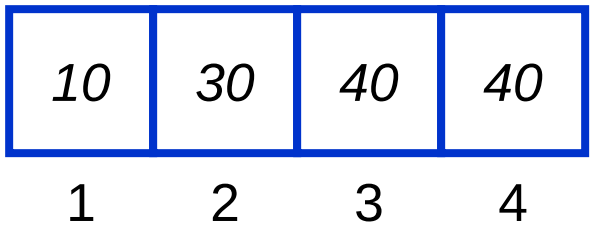


i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

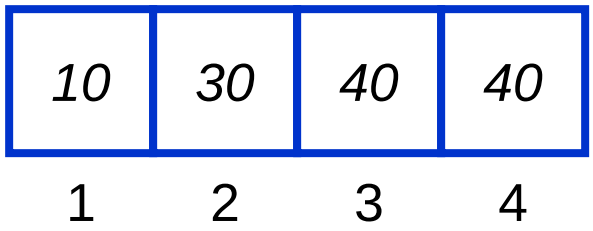


i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

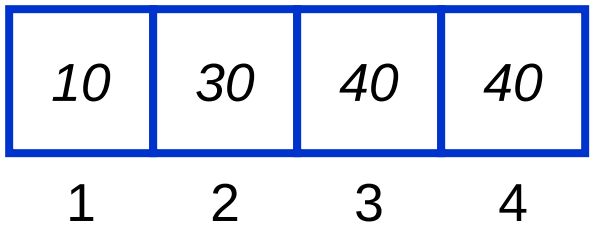


i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

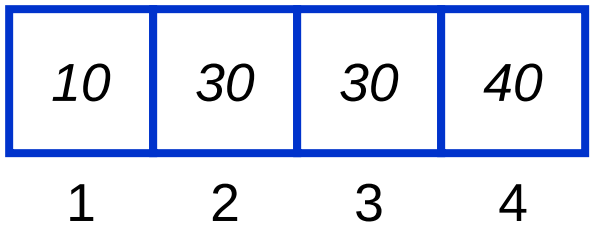


i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 40



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

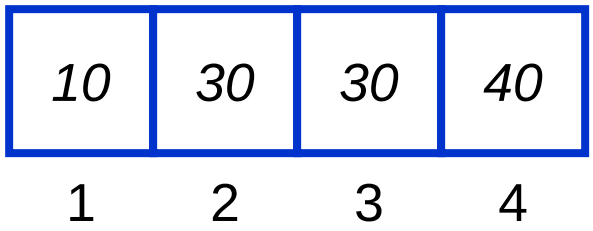


i = 4	j = 2	key = 20
A[j] = 30		A[j+1] = 30



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

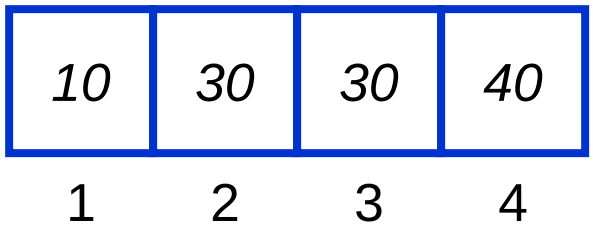


i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 30



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort

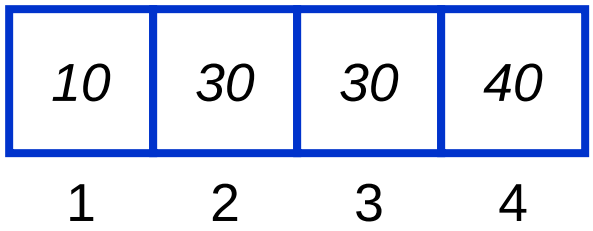


i = 4 j = 1 key = 20
A[j] = 10 A[j+1] = 30



```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

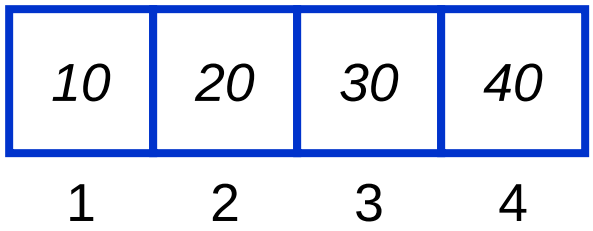
Illustration of Insertion Sort



i = 4 j = 1 key = 20
A[j] = 10 A[j+1] = 30

```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

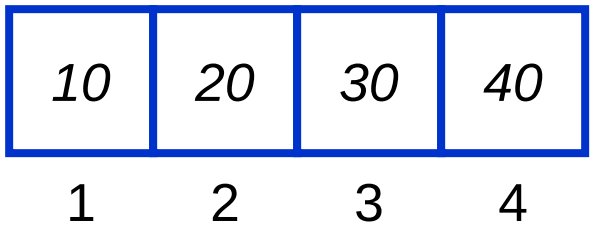
Illustration of Insertion Sort



i = 4 j = 1 key = 20
A[j] = 10 A[j+1] = 20

```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Illustration of Insertion Sort



i = 4 j = 1 key = 20
A[j] = 10 A[j+1] = 20

```
InsertionSort(A, n) {
   for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
   }
}
```

Done!

Running Time

Number of primitive steps that are executed

Time complexity of insertion sort

- Worst case: input in descending order
  - Count the number of comparisons
    - for i=2, no. of inner loop iteration : 1
    - for i=3, no. of inner loop iteration : 2
    - .....
    - for i=n, no. of inner loop iteration : (n-1)
  - Total number of comparisons

    $[1 + 2 + \ldots + (n - 1)] = (n * (n - 1)) / 2$

    $= (n^2 - n) / 2$

    $< n^2$

Therefore, worst case time complexity is $O(n^2)$

Time complexity of insertion sort

- Best case: input in sorted order
  - Count the number of comparisons
  - ??

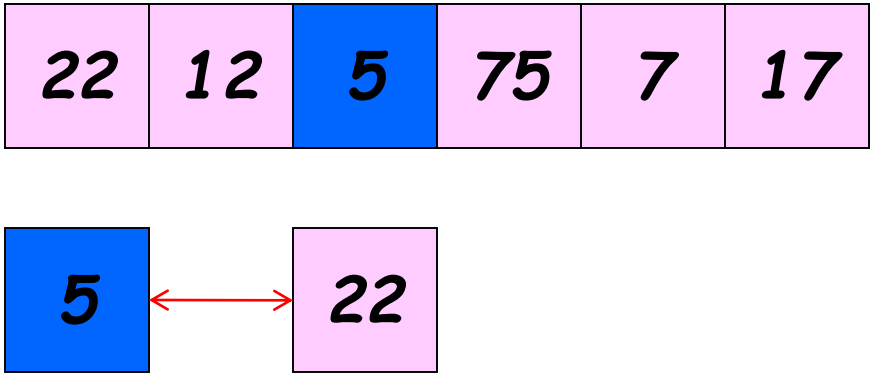
Selection Sort

Logic:

1) Find the smallest element in the array and exchange it with the element in the first position of the array.
2) Find the second smallest element and exchange it with the element in the second position of the array.
3) Apply same logic until the entire array is sorted.

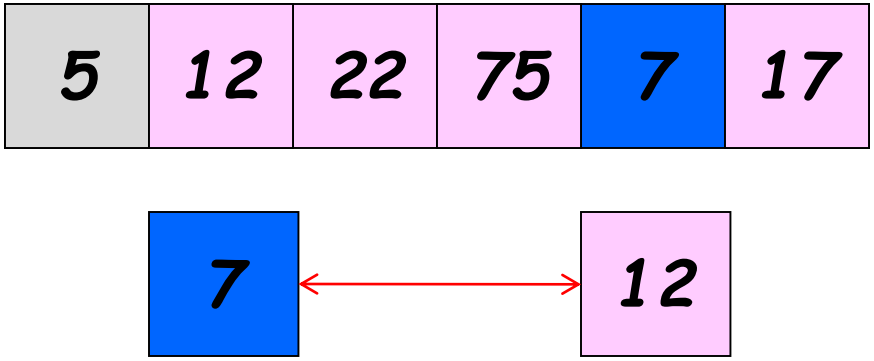
Selection Sort illustration

Search the entire list for the smallest element and swaps it into 1st position of the array



Selection Sort illustration

Search for the second smallest and swaps it into 2nd position of the array



Continue … for 3rd, 4th position and so on until the entire array is sorted

Selection sort algorithm

Input: An array **A[1…N]**
Output: The array **A** sorted in ascending order

Algorithm SelectionSort (A, N):

```
for i = 1 to N – 1  {
    smallest = i
    for j = i + 1 to N do {
            if (A[j] < A[smallest])
                smallest = j
     }
     temp = A[i]
     A[i] = A[smallest]
     A[smallest] = temp
}
```

Time complexity of the selection sort

1) Count the number of comparisons

- for 1st iteration of the outer loop, no. of comparisons in the inner loop: N – 1
- for 2nd iteration of the outer loop, no. of comparisons in the inner loop: N – 2
- ...
- for Nth iteration of the outer loop, no. of comparisons in the inner loop: 1

2) Total number of comparisons (T1)

$$(N - 1) + (N - 2) + \ldots + 2 + 1 = (N * (N - 1)) / 2$$

$$= (N^2 - N) / 2$$

Time complexity of the selection sort

1) Count the number of exchanges
   - for 1st iteration of the outer loop: 1
   - for the 2nd iteration of the outer loop: 1
   - ...
   - for the $(N-1)^{th}$ iteration of the outer loop: 1

2) Total number of exchanges (T2) = N – 1

Therefore, time complexity = T1 + T2 = $(N^2-N)/2 + N - 1$
= $O(N^2)$

Selection sort vs Insertion sort

- For almost sorted objects, insertion sort is a good choice.
- For unsorted objects, selection sort can be a better choice than the insertion sort.

Bubble sort algorithm

Input: An array **A[1…N]**
Output: The array **A**

Algorithm BubbleSort (A, N):

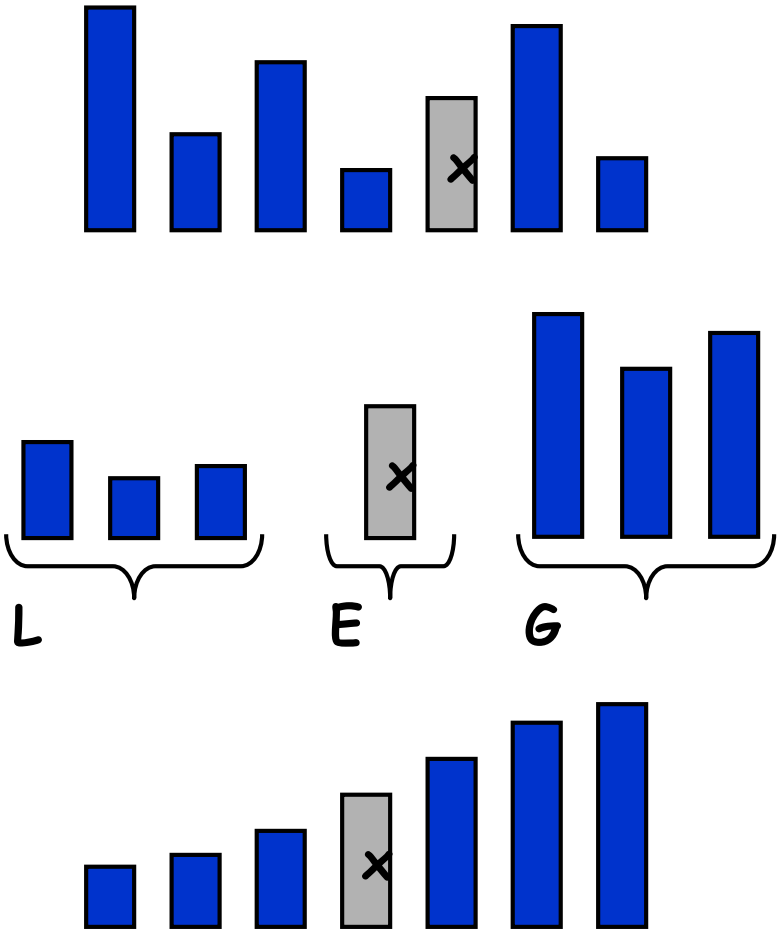
```
while (N>2) {
    for i = 1 to N-1 {
        if A[i] > A[i+1] then swap( A[i], A[i+1] )
    }
N=N-1
}
```

Task: Analyze its time complexity.

Quicksort

Quicksort

- Quicksort is a sorting algorithm based on the divide-and-conquer paradigm:
  - Divide: pick a random element ***x*** (called pivot) and partition ***S*** into
    - ***L*** elements less than ***x***
    - ***E*** elements equal ***x***
    - ***G*** elements greater than ***x***
  - Recur: sort ***L*** and ***G***
  - Conquer: join ***L***, ***E*** and ***G***



Partition

- We partition an input sequence as follows:
  - We remove each element **y** from **S**
  - We insert **y** into **L**, **E** or **G**, depending on the result of the comparison with the pivot **x**
- Each insertion and removal is at the beginning or at the end of a sequence, and it takes **O**(1) time.
- Thus, the partition step of quick-sort takes **O**(**n**) time.

Input sequence **S**, position **p** of pivot
Output subsequences **L**, **E**, **G** of the elements of **S** less than, equal to, or greater than the pivot.

```
Algorithm partition(S, p)
    L, E, G ← empty sequences
    x ← S.remove(p)
    while ¬S.isEmpty()
        y ← S.remove(S.first())
        if y < x
            L.insertLast(y)
        else if y = x
            E.insertLast(y)
        else
            G.insertLast(y)
    return L, E, G
```

Partition (cont.)

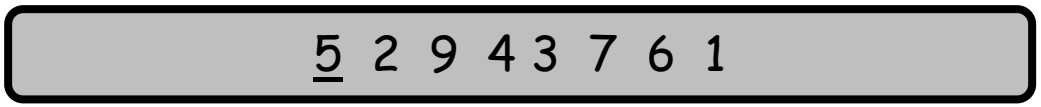
```
int partition(A, l, r, p)  //p: pivot index
   pivotElement = A[p];
   swap(A[p], A[r])  //move pivot element to last
   po = l;   //po: partitioning index
   for i = l to r-1 {
       if (A[i] < pivotElement) {
              swap(A[i], A[po]);
              po = po + 1;
       }
   }
   swap(A[po], A[r];
   return po;
```

Quicksort algorithm

```
quicksort(A, l, r) //l: left index, r: right index
{
   if (l < r) {
         q = partition(A, l, r, p);
         quicksort(A, l, q-1);
         quicksort(A, q+1, r);
     }
}
```

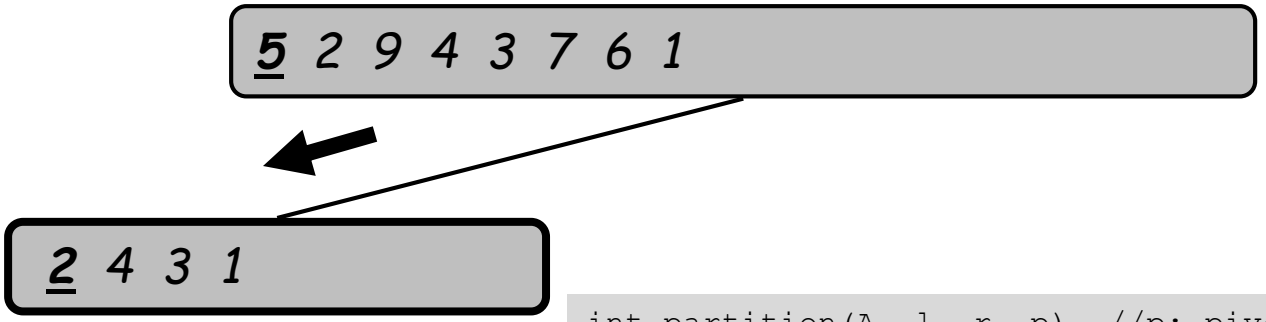
Example

- Pivot selection



Example

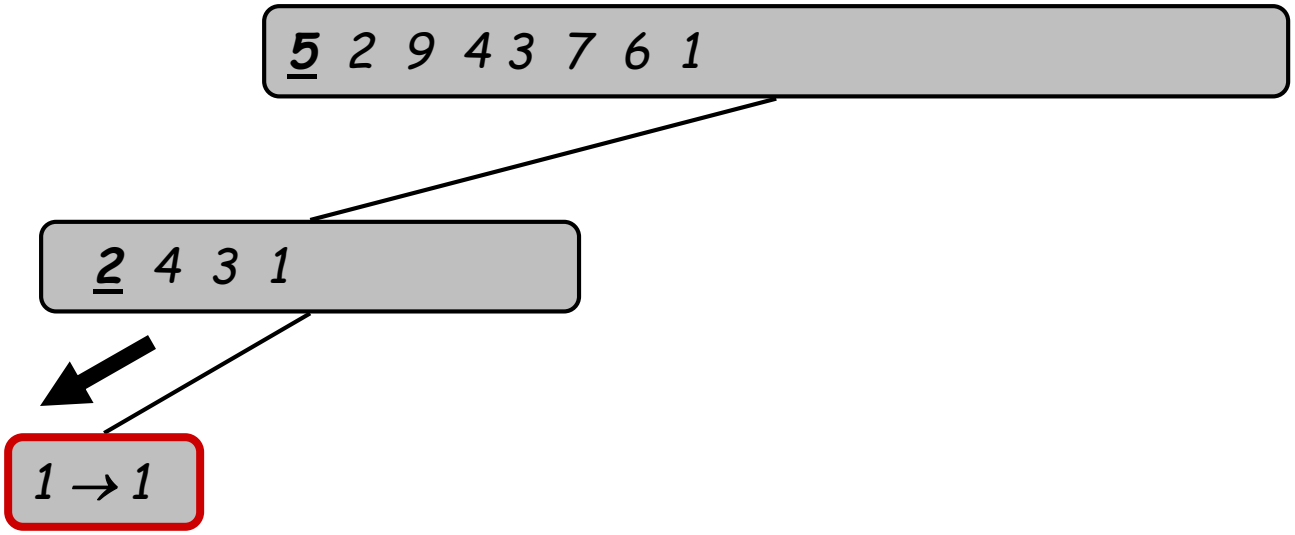
- Partition, recursive call, pivot selection




```
int partition(A, l, r, p)  //p: pivot index
    pivotElement = A[p];
    swap(A[p], A[r])  //move pivot element to last
    po = l;   //po: partitioning index
    for i = l to r-1 {
          if (A[i] < pivotElement) {
                swap(A[i], A[po]);
                pos = pos + 1;
          }
    }
    swap(A[po], A[r];
    return po;
```

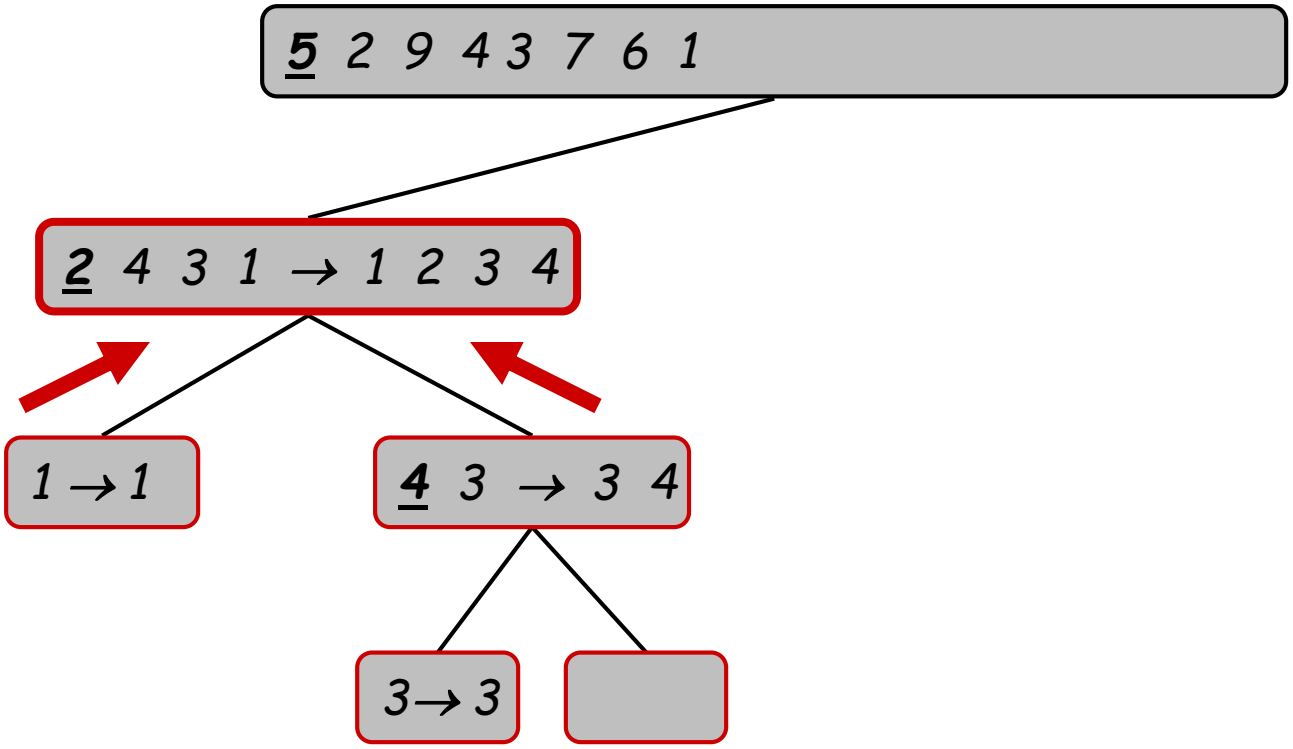
Example

- Partition, recursive call, base case



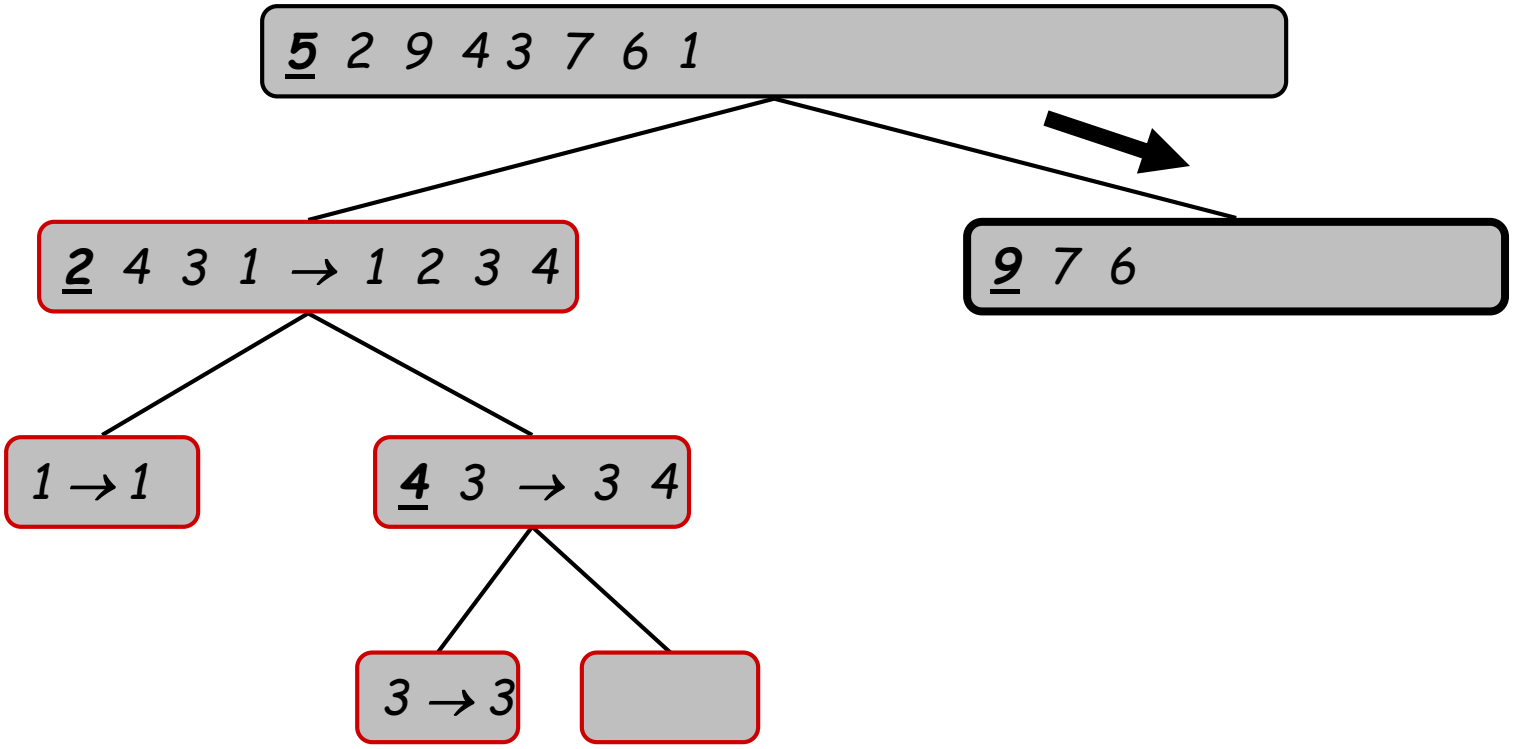
Example

- Recursive call, …, base case, join



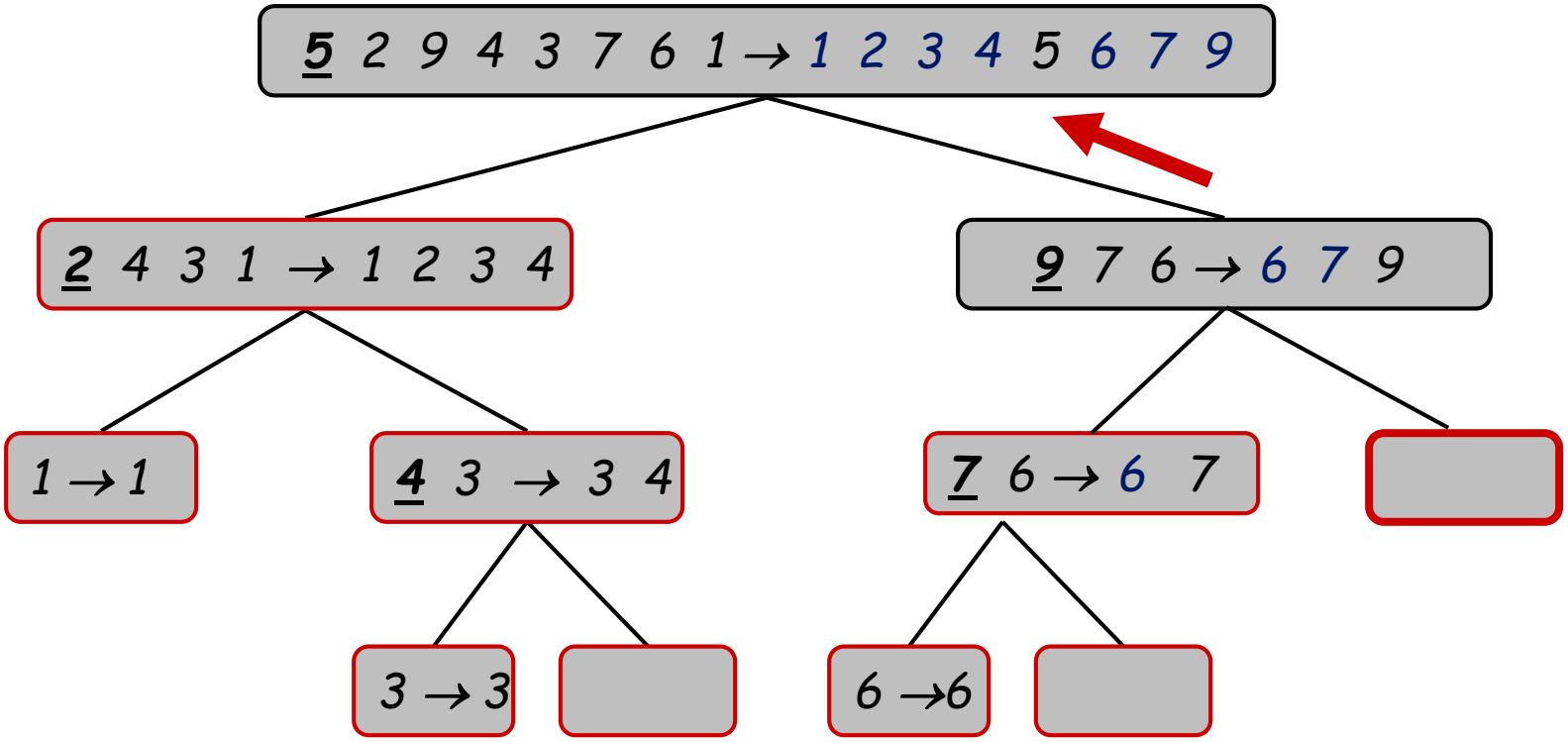
Example

- Recursive call, pivot selection



Example

Partition,…, recursive call, base case, Join

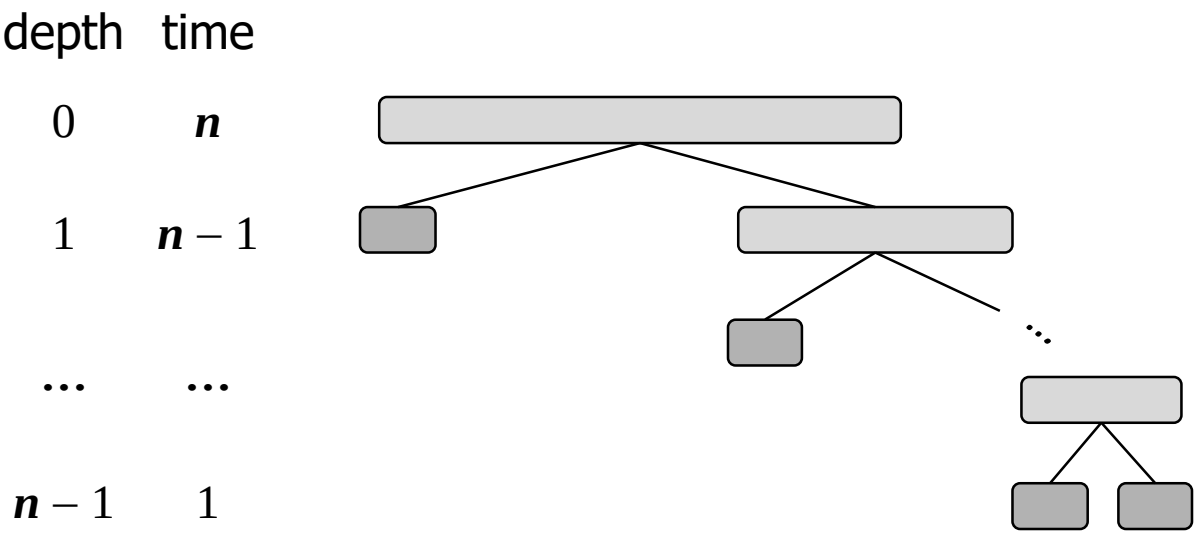


Worst-case Running Time

- The worst case for quicksort occurs when the pivot is the unique minimum or maximum element and the list is sorted
- One of ***L*** and ***G*** has size ***n*** - 1 and the other has size 0
- The running time is proportional to the sum

$$n + (n - 1) + \ldots + 2 + 1$$

- Thus, the worst-case running time of quick-sort is $O(n^2)$



Analyzing Quicksort

- In the worst case: when the list is already sorted

  $T(1) = \Theta(1)$

  $T(n) = T(n - 1) + \Theta(n)$

- Complexity: $T(n) = \Theta(n^2)$

Analyzing Quicksort

- In the average/best case: when list contains random elements

  $T(1) = \Theta(1)$

  $T(n) = 2T(n/2) + \Theta(n)$

- Complexity: $T(n) = \Theta(n \log n)$

MergeSort

Merge-sort on an input sequence S with n elements consists of three steps:

- Divide: partition S into two sequences S_1 and S_2 of about $n/2$ elements each
- Recur: recursively sort S_1 and S_2
- Conquer: merge S_1 and S_2 into a unique sorted sequence

```
Algorithm mergeSort(S)
    Input sequence S with n
        elements
    Output sequence S sorted

    if S.size() > 1
        (S1, S2) ← partition(S, n/2)
        mergeSort(S1)
        mergeSort(S2)
        S ← merge(S1, S2)
```

Merging Two Sorted Sequences

- The conquer step of merge-sort consists of merging two sorted sequences ***A*** and ***B*** into a sorted sequence ***S*** containing the union of the elements of ***A*** and ***B***
- Merging two sorted sequences, each with ***n***/2 elements takes ***O***(***n***) time.

```
Algorithm merge(A, B)
Input sequences A and B with n/2 elements each
Output sorted sequence S of A ∪ B

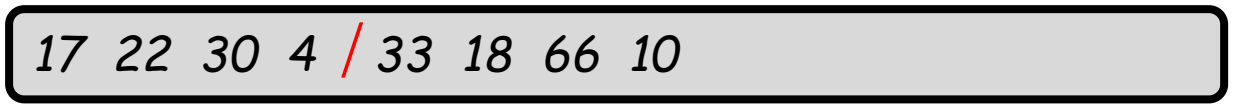
    S ← empty sequence
    while ¬A.isEmpty() ∧ ¬B.isEmpty()
        if A.first().element() < B.first().element()
            S.insertLast(A.remove(A.first()))
        else
            S.insertLast(B.remove(B.first()))
    while ¬A.isEmpty()
        S.insertLast(A.remove(A.first()))
    while ¬B.isEmpty()
        S.insertLast(B.remove(B.first()))
    return S
```

MergeSort

```
MergeSort(S, left, right) {
    if (left < right) {  //left index and right index of array of elements
            mid = floor((left + right) / 2);
            MergeSort(S, left, mid);
            MergeSort(S, mid+1, right);
            Merge(S, left, right);
    }
}
```

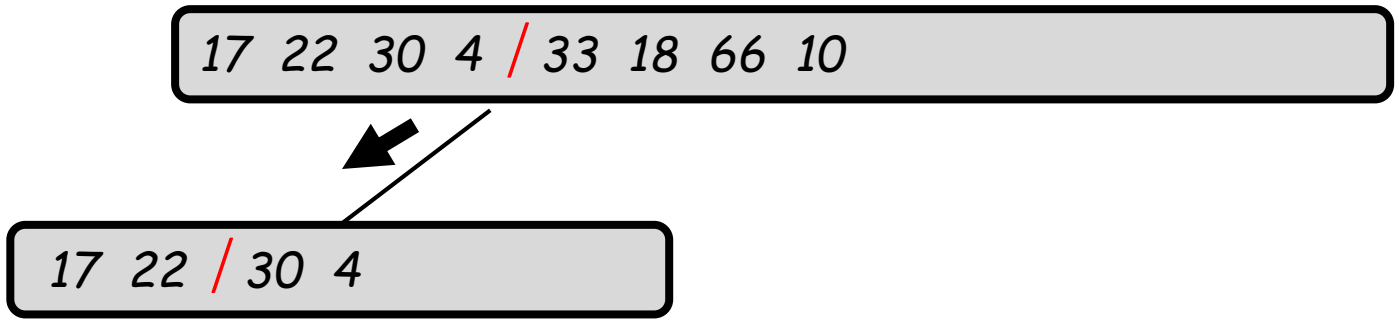
Example

Partition



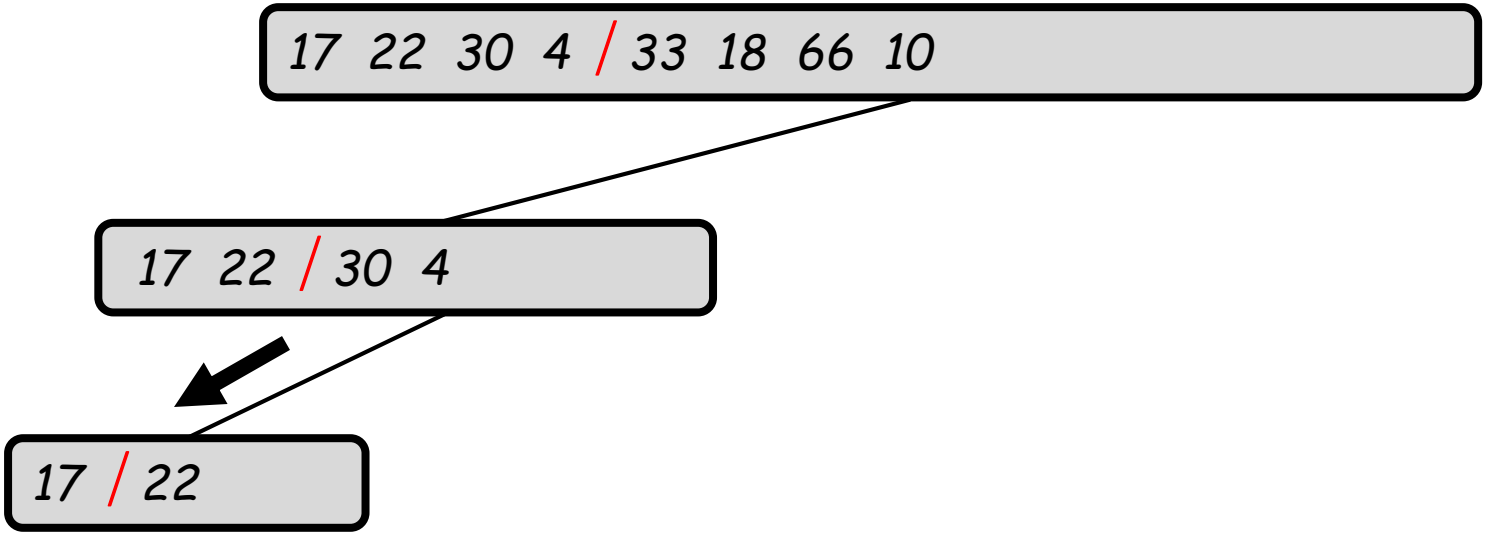
Example (cont.)

Recursive call, partition



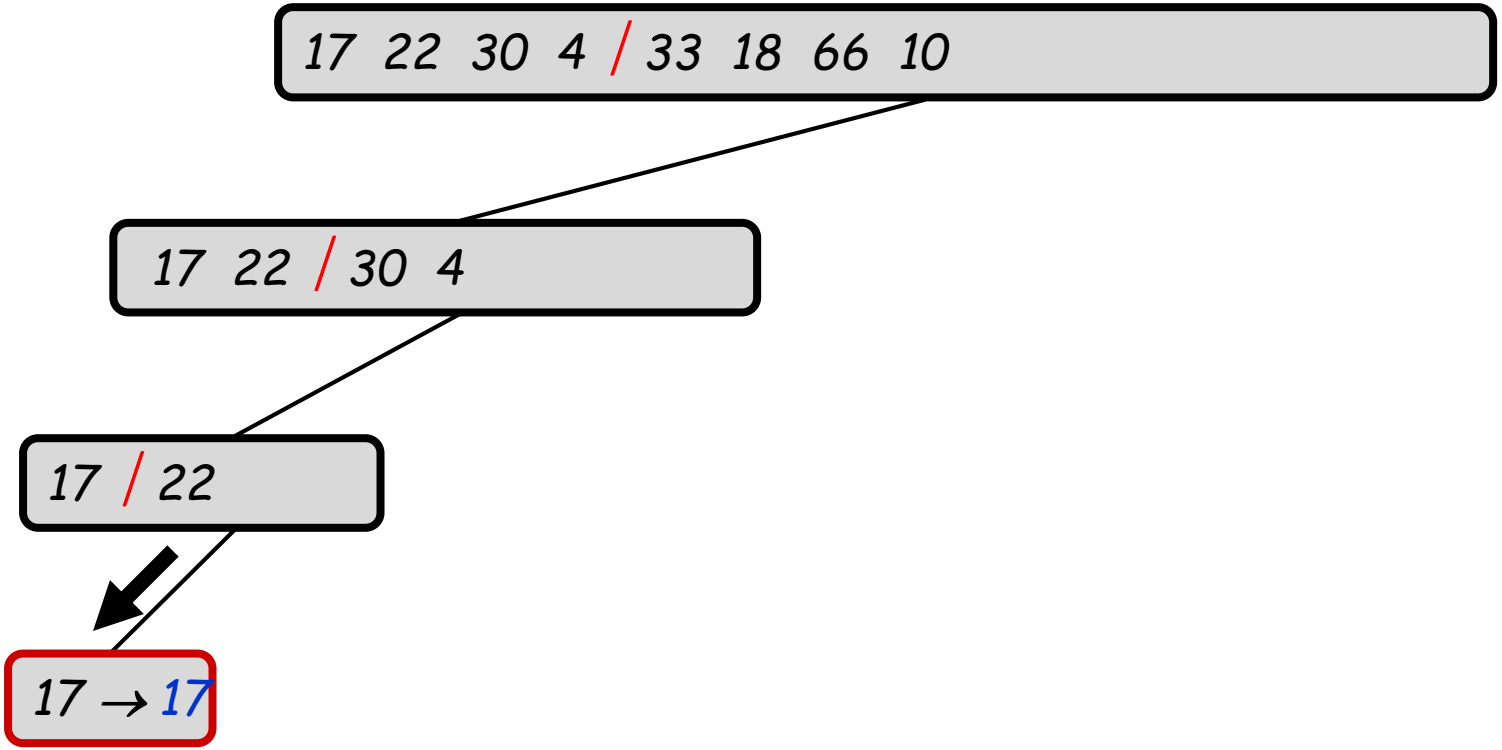
Example (cont.)

Recursive call, partition



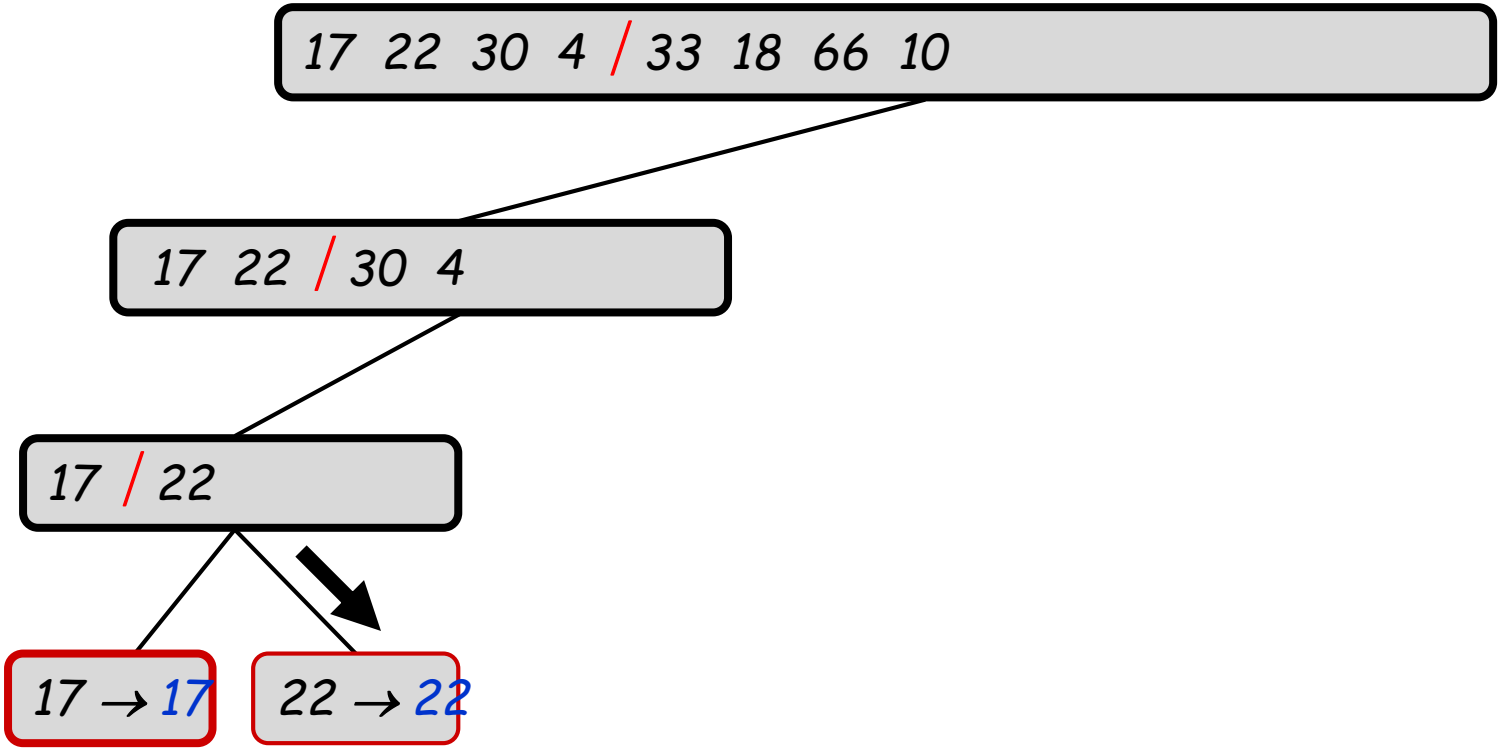
Example (cont.)

Recursive call, base case



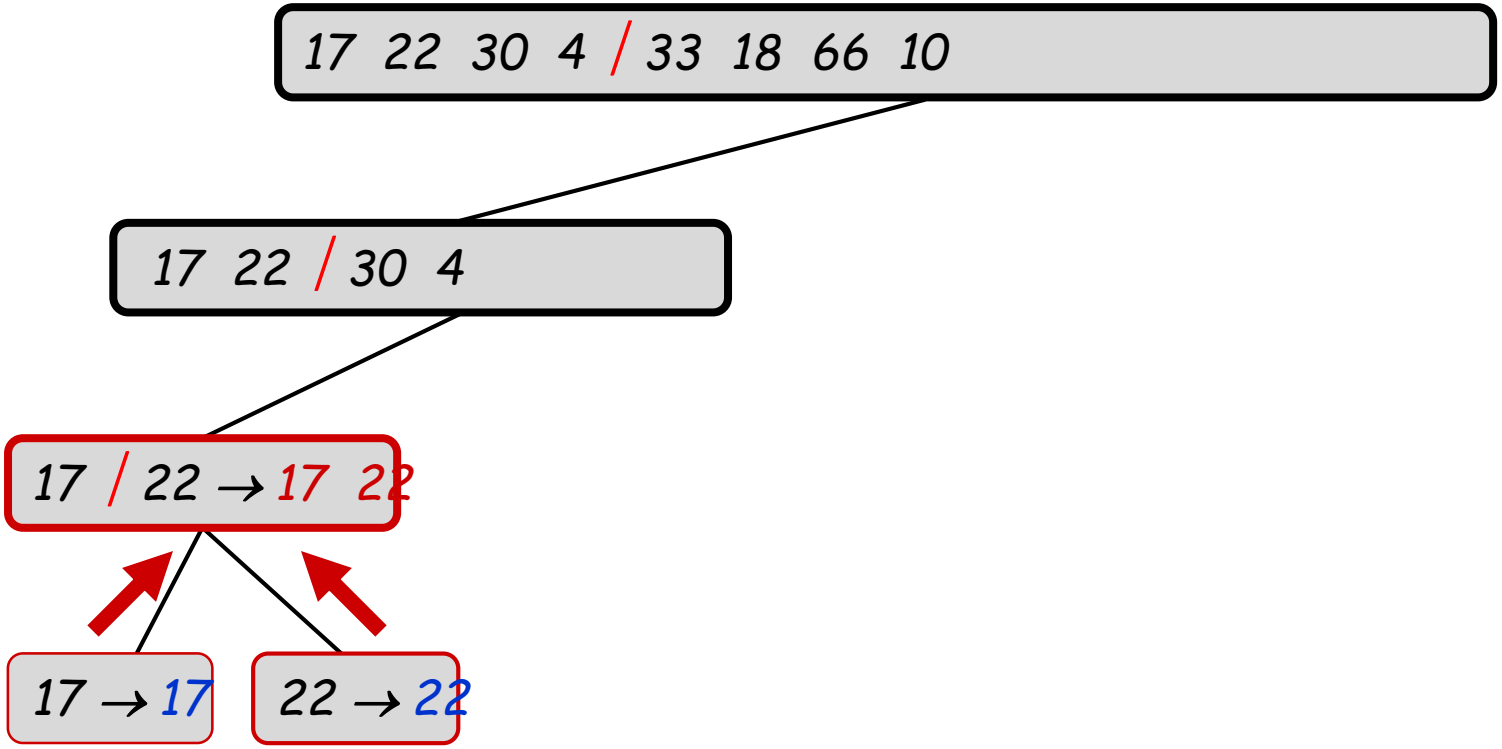
Example (cont.)

Recursive call, base case



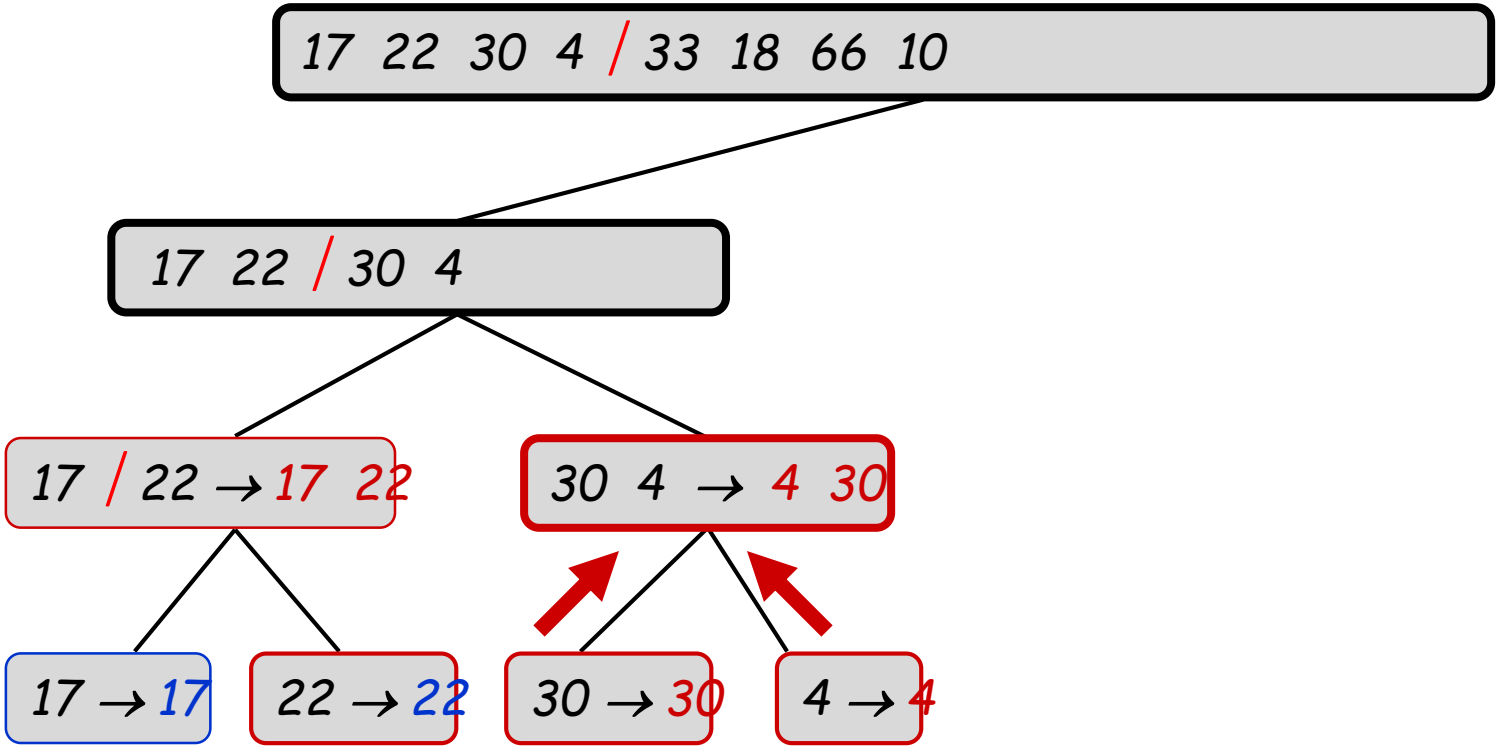
Example (cont.)

Merge



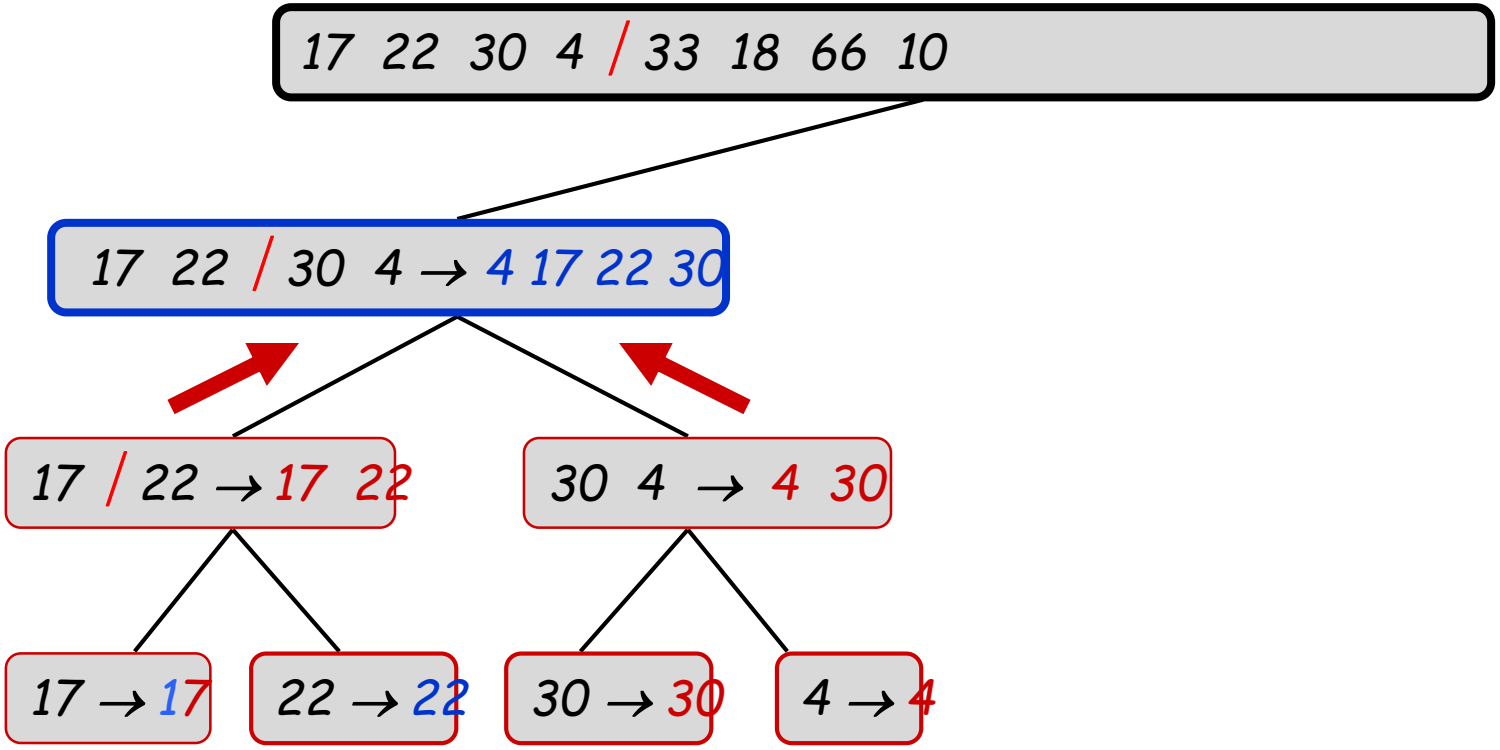
Example (cont.)

Recursive call, …, base case, merge



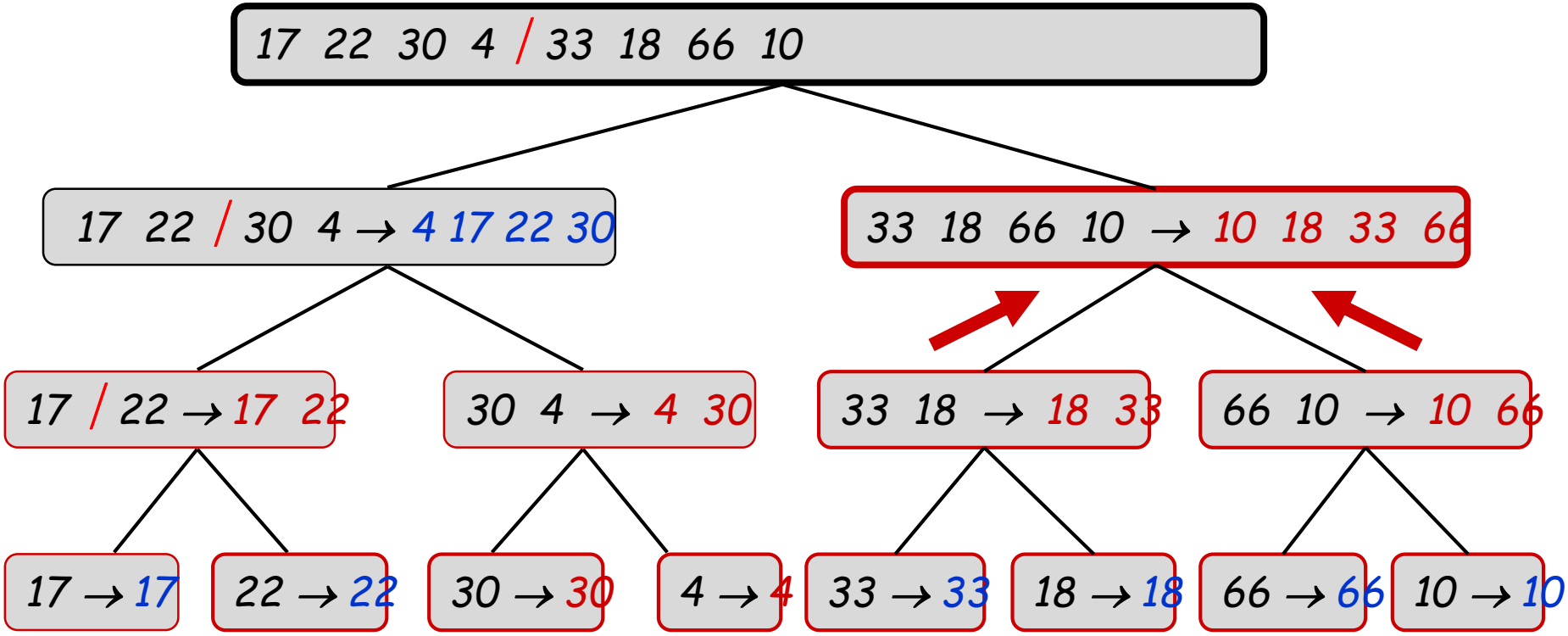
Example (cont.)

Merge



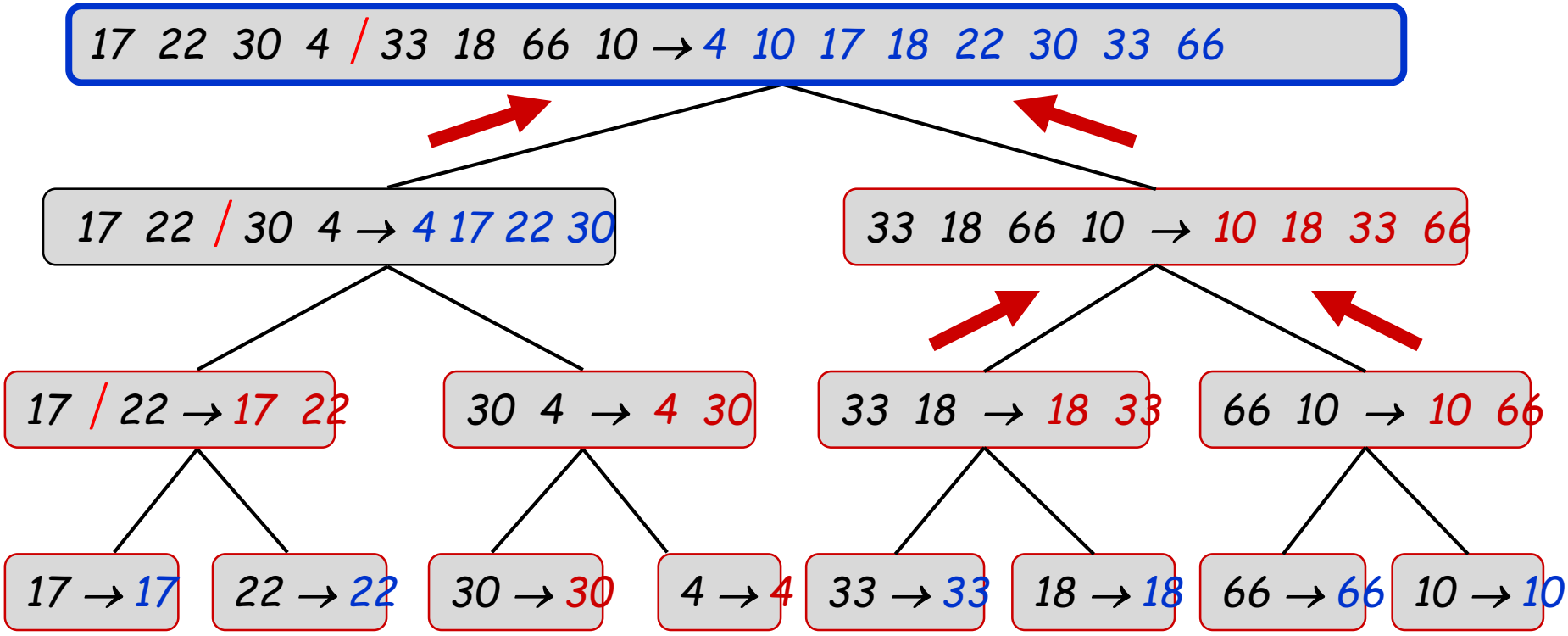
Example (cont.)

Recursive call, …, merge, merge



Example (cont.)

Merge



Task

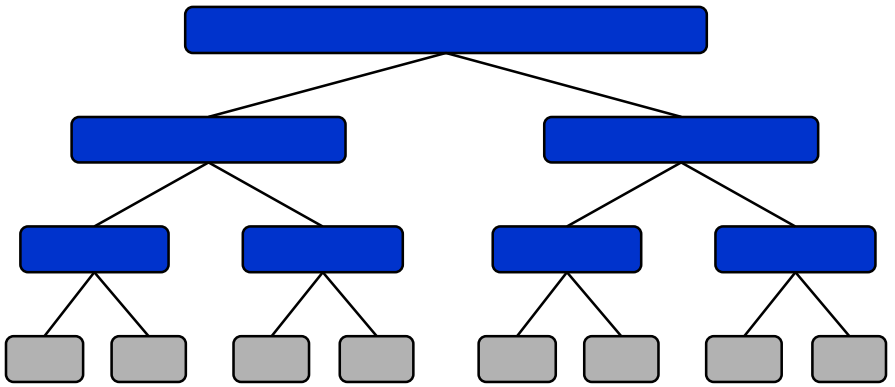
Run MergeSort() for following array

$A = \{10, 50, 7, 62, 17, 41, 88, 3, 25, 39\}$

Analysis of MergeSort

- The height ***h*** of the merge-sort tree is ***O***(log ***n***)
  - at each recursive call we divide in half the sequence,
- The overall amount of work done at the nodes of depth ***i*** is ***O***(***n***)
  - we partition and merge 2^i sequences of size $n/2^i$
  - we make 2^{i+1} recursive calls
- Thus, the total running time of merge-sort is ***O***(***n*** log ***n***)

depth	#Ts	size
0	1	n
1	2	$n/2$
i	2^i	$n/2^i$
…	…	…



Recommended Reading

- Goodrich et al - Ch #10
- CLRS (3rd ed.) -Ch #4, #7